TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG GIÁM SÁT ĐẤT VÀ KHÔNG KHÍ, ĐIỀU KHIỂN TƯỚI TIÊU THEO THỜI GIAN THỰC

TÊN LỚP HỌC PHẦN: PHÁT TRIỂN ỨNG DỤNG IOT

MÃ HỌC PHẦN: 2025-2026.2.TIN4024.009

GIẢNG VIÊN HƯỚNG DẪN: TS. LÊ HỮU BÌNH

Huế, Ngày 10 Tháng 04 Năm 2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn sâu sắc nhất đến Khoa Công nghệ Thông tin – Trường Đại học Khoa học – Đại học Huế, nơi đã tạo điều kiện thuận lợi và đồng hành cùng em trong suốt quá trình học tập, đặc biệt là hoàn thành bài tiểu luận này. Môi trường học thuật chuyên nghiệp tại Khoa đã giúp em không ngừng phát triển kiến thức chuyên môn, đồng thời rèn luyện những kỹ năng thực tiễn thiết yếu cho con đường sự nghiệp sắp tới.

Đặc biệt, em muốn bày tỏ lòng biết ơn chân thành đến TS. Lê Hữu Bình, người thầy đã tận tâm hướng dẫn, chia sẻ kinh nghiệm quý báu và truyền cảm hứng cho em từ những bước đầu tiên của đề tài cho đến khi hoàn thành. Thầy không chỉ giúp em vượt qua những thách thức kỹ thuật trong quá trình phát triển hệ thống IoT giám sát môi trường nông nghiệp, mà còn định hướng cho em cách tiếp cận bài toán một cách khoa học và có trách nhiệm.

Cuối cùng, em xin gửi lời tri ân đến toàn thể quý thầy cô trong Khoa, những người đã tận tâm truyền đạt kiến thức, tạo nền tảng vững chắc để em có thể tự tin bước vào thế giới công nghệ. Em sẽ luôn ghi nhớ và trân trọng những giá trị mà quý thầy cô đã trao tặng.

Xin trân trọng cảm ơn!

Huế, tháng 04 năm 2026

DANH MỤC CÁC TỪ VIẾT TẮT

Ký hiệu	Giải thích
ADC	Analog-to-Digital Converter — Bộ chuyển đổi tương tự sang số
DHT	Digital Humidity and Temperature — Cảm biến nhiệt độ và độ ẩm số
GPIO	General Purpose Input/Output — Chân vào/ra đa mục đích
IoT	Internet of Things — Internet vạn vật
JSON	JavaScript Object Notation — Định dạng dữ liệu văn bản nhẹ
MQTT	Message Queuing Telemetry Transport — Giao thức truyền thông pub/sub
SSL/TLS	Secure Sockets Layer / Transport Layer Security — Giao thức bảo mật tầng truyền tải
I2C	Inter-Integrated Circuit — Giao thức truyền thông nối tiếp

Mục lục

CHƯƠNG 1: ĐẶT VẤN ĐỀ **1**
1.1 Bối cảnh và tính cấp thiết của đề tài 1
1.2 Mục tiêu của đề tài 1
1.3 Phạm vi và đối tượng nghiên cứu 2

CHƯƠNG 2: NỘI DUNG THỰC HIỆN **2**
2.1 Thu thập dữ liệu môi trường (DHT22, độ ẩm đất và cường độ ánh sáng) . 2
2.2 Điều khiển máy bơm qua Relay và cảnh báo Buzzer 3
2.3 Truyền dữ liệu qua MQTT (HiveMQ Cloud, TLS) 4
2.4 Xử lý dữ liệu bằng Node-RED 5
2.5 Hiển thị dữ liệu trên Dashboard 6
2.6 Cảnh báo môi trường 7
2.7 Điều khiển máy bơm từ xa qua MQTT 7
2.8 Duy trì kết nối và đồng bộ trạng thái bơm 8
2.9 Tự động tưới theo ngưỡng độ ẩm đất 9

CHƯƠNG 3: THIẾT KẾ MẠCH IoT **9**
3.1 Thành phần phần cứng 9
3.2 Kết nối phần cứng (GPIO) 10
3.2.1 Mạch vật lý — ảnh mô phỏng Wokwi (tương đương mạch thực tế đã nối) 11
3.2.2 Sơ đồ mạch nguyên lý (EasyEDA) 12
3.3 Giải thích các linh kiện chính 13
3.3.1 Vi điều khiển ESP32 DevKit C v4 13
3.3.2 Cảm biến nhiệt độ và độ ẩm không khí DHT22 14
3.3.3 Cảm biến cường độ ánh sáng BH1750 15
3.3.4 Biến trở (mô phỏng cảm biến độ ẩm đất) 16
3.3.5 Module Relay điều khiển máy bơm 17
3.3.6 Còi Buzzer cảnh báo 18
3.4 Luồng hoạt động tóm tắt theo chức năng 18

CHƯƠNG 4: NGUYÊN LÝ HOẠT ĐỘNG **19**
4.1 Sơ đồ luồng hệ thống (kiến trúc ba tầng) 19
4.2 Luồng giám sát: cảm biến → MQTT → Dashboard 19
4.3 Luồng điều khiển: Dashboard → MQTT → ESP32 (Relay điều khiển bơm) 20
4.4 Luồng đồng bộ trạng thái bơm và giới hạn thời gian chạy 20
4.5 Luồng tưới tự động theo ngưỡng độ ẩm đất (Node-RED) 20
4.6 Luồng duy trì kết nối Wi-Fi và MQTT 21

CHƯƠNG 5: CÀI ĐẶT HỆ THỐNG **21**
5.1 Chuẩn bị công cụ và môi trường phát triển 21
5.2 Khởi tạo sơ đồ mạch mô phỏng và cấu hình phần cứng 21
5.3 Triển khai và vận hành Firmware cho ESP32 22

5.3.1 Cài đặt mã nguồn và thư viện 22
5.3.2 Cấu hình thông số hệ thống 22
5.3.3 Vận hành mô phỏng 22
5.4 Thiết lập hệ thống điều khiển và giao diện Node-RED 23
5.4.1 Lệnh cài đặt nền tảng 23
5.4.2 Triển khai luồng xử lý và giao diện Dashboard 23
5.5 Vận hành và kiểm thử tổng thể 23

CHƯƠNG 6: KẾT QUẢ THỰC HIỆN 24
6.1 Kết quả mô phỏng phần cứng trên Wokwi 24
6.2 Kết quả truyền nhận dữ liệu qua Serial Monitor 24
6.3 Thiết lập luồng xử lý dữ liệu trên Node-RED 28
6.4 Giao diện giám sát và điều khiển Dashboard 28
6.5 Phân tích luồng hoạt động thực tế 29

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 32
7.1 Các công việc đã hoàn thành 32
7.2 Công việc chưa thực hiện và hạn chế 32
7.3 Hướng phát triển tiếp theo 32

TÀI LIỆU THAM KHẢO
Phụ lục A. Mã nguồn chương trình ESP32 . . . . . .
Phụ lục B. Mã nguồn Node-RED (JSON Flow) . . . . . .
Phụ lục C. Mã nguồn sơ đồ Wokwi (JSON) . . . . . .

CHƯƠNG 1: ĐẶT VẤN ĐỀ

1.1 Bối cảnh và tính cấp thiết của đề tài

Trong thời đại nông nghiệp 4.0, việc tối ưu hóa quy trình chăm sóc cây trồng bằng công nghệ là một nhu cầu cấp bách. Các phương pháp tưới tiêu truyền thống thường dựa trên cảm tính hoặc lịch trình cố định, không phản ánh đúng trạng thái thực tế của đất và môi trường xung quanh, dẫn đến lãng phí tài nguyên nước hoặc gây hại cho sự phát triển của cây.

Hệ thống IoT (Internet of Things) giám sát môi trường nông nghiệp ra đời nhằm giải quyết vấn đề này. Bằng cách kết hợp các cảm biến đo đạc tức thời và khả năng điều khiển từ xa thông qua mạng internet, người quản lý có thể theo dõi chính xác tình trạng vườn trồng mọi lúc mọi nơi. Việc triển khai một hệ thống giám sát đất, không khí và ánh sáng kết hợp với cơ chế tưới tự động không chỉ giúp tiết kiệm công sức mà còn đảm bảo môi trường sinh trưởng tốt nhất cho cây trồng.

1.2 Mục tiêu của đề tài

Dự án tập trung vào việc thiết kế và triển khai một hệ thống IoT hoàn chỉnh với các mục tiêu cụ thể sau:

- Xây dựng hệ thống thu thập dữ liệu môi trường (Nhiệt độ, Độ ẩm không khí, Độ ẩm đất và Cường độ ánh sáng) theo thời gian thực.
- Triển khai giao thức truyền thông MQTT bảo mật qua TLS để kết nối thiết bị với nền tảng đám mây.
- Thiết kế giao diện Dashboard giám sát trực quan và tích hợp các nút điều khiển máy bơm, công tắc chuyển đổi chế độ tưới thông minh.
- Xây dựng cơ chế cảnh báo tại chỗ (Buzzer) và trên ứng dụng (Toast UI) khi các chỉ số môi trường vượt ngưỡng an toàn.

1.3 Phạm vi và đối tượng nghiên cứu

Đối tượng nghiên cứu:

- Vi điều khiển ESP32 và các loại cảm biến (DHT22, BH1750, Potentiometer).
- Giao diện lập trình luồng Node-RED và giao thức MQTT (HiveMQ Cloud).

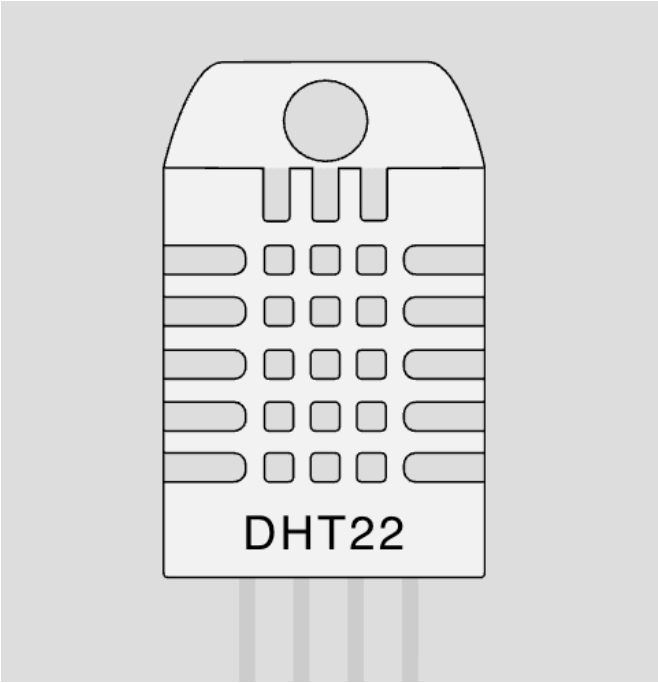
Phạm vi nghiên cứu:

- Hệ thống được mô phỏng trên nền tảng Wokwi với đầy đủ các thành phần phần cứng và kết nối mạng thực tế.
- Tập trung vào logic điều khiển tưới dựa trên độ ẩm đất và các cảnh báo dựa trên ngưỡng nhiệt độ, ánh sáng.

CHƯƠNG 2: NỘI DUNG THỰC HIỆN

2.1 Thu thập dữ liệu môi trường (DHT22, độ ẩm đất và cường độ ánh sáng)

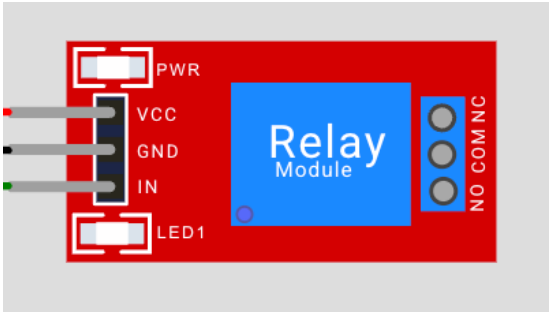
Firmware định kỳ 5 giây (`PUBLISH_INTERVAL`) gọi `readSensors()`: đọc nhiệt độ và độ ẩm không khí qua thư viện DHT trên GPIO `DHT_PIN` (4), đọc độ ẩm đất thô qua `analogRead(SOIL_PIN)` với `SOIL_PIN` = 34 (ADC 12-bit, 0–4095), và đọc cường độ ánh sáng qua cảm biến BH1750 giao tiếp I2C (GPIO 21-22). Nếu DHT trả NaN, giá trị được thay bằng số ngẫu nhiên trong khoảng an toàn để hệ thống không dừng (`main.cpp`). DHT22 đo nhiệt độ và độ ẩm không khí theo giao tiếp 1-wire với độ chính xác cao. BH1750 là cảm biến đo cường độ ánh sáng giao tiếp I2C, phù hợp cho giám sát điều kiện quang hợp của cây trồng. Potentiometer mô phỏng cảm biến độ ẩm đất: giá trị ADC được quy đổi sang phần trăm theo công thức `map(raw, 0, 4095, 0, 100)`.



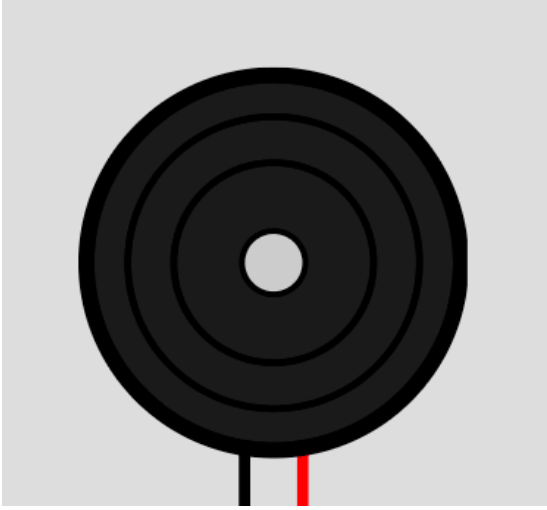
Hình 1: Cảm biến DHT22 đo đạc thông số nhiệt độ và độ ẩm không khí

2.2 Điều khiển máy bơm qua Relay và cảnh báo Buzzer

GPIO `PUMP_RELAY_PIN` (26) điều khiển Relay module kết nối máy bơm nước. GPIO `BUZZER_PIN` (27) dùng PWM phát âm thanh cảnh báo trong hàm `beepAlert()`: khi độ ẩm đất < 30%, phát bíp 2 lần (ngưỡng khô); khi độ ẩm đất < 20%, phát bíp 5 lần liên tục (ngưỡng nguy hiểm) (**`main.cpp`**).



Hình 2: Module Relay điều khiển máy bơm nước



Hình 3: Còi Buzzer phát âm thanh cảnh báo

2.3 Truyền dữ liệu qua MQTT (HiveMQ Cloud, TLS)

MQTT (Message Queuing Telemetry Transport) là giao thức truyền thông nhẹ, được thiết kế đặc biệt cho các thiết bị IoT có băng thông thấp, năng lượng hạn chế hoặc kết nối không ổn định. MQTT hoạt động theo mô hình publisher/subscriber (pub/sub), trong đó thiết bị gửi (publisher) thực hiện publish dữ liệu lên một topic trên MQTT Broker, và các thiết bị nhận (subscriber) sẽ subscribe vào các topic tương ứng để nhận dữ liệu. Mô hình này tách biệt người gửi và người nhận, giúp hệ thống dễ mở rộng.

MQTT sử dụng cơ chế QoS (Quality of Service) với ba mức: QoS 0 (At most once) — message được publish một lần, không xác nhận; QoS 1 (At least once) — gửi ít nhất một lần, có xác nhận, đảm bảo message đến; QoS 2 (Exactly once) — đảm bảo message đến đúng một lần, phù hợp cho các ứng dụng yêu cầu độ tin cậy cao. Trong đề tài này, QoS 0 được sử dụng cho cả việc publish dữ liệu cảm biến và điều khiển, phù hợp với tần suất dữ liệu không quá cao.

HiveMQ Cloud là dịch vụ MQTT Broker dạng cloud do HiveMQ cung cấp, hỗ trợ giao thức bảo mật TLS (Transport Layer Security) trên cổng 8883, đảm bảo dữ liệu truyền giữa thiết bị và broker được mã hóa end-to-end. HiveMQ Cloud có gói miễn phí (Free Tier) cho phép kết nối tối đa 100 thiết bị, đủ cho các ứng dụng demo và học tập.

Cấu hình MQTT trong đề tài:

- Broker: `b1c8f2cc5cd4416fb74b671a407dc0ff.s1.eu.hivemq.cloud`
- Port: **8883** (TLS)
- Protocol: MQTT v3.1.1
- Topic gửi dữ liệu cảm biến: `iot/farm/env`
- Topic nhận lệnh điều khiển: `iot/farm/control`
- Topic nhận cấu hình: `iot/farm/config`

Lưu ý: các topic được đặt theo chuẩn để dễ mở rộng. Trên ESP32, client dùng `WiFiClientSecure` với TLS (cổng `MQTT_PORT` = 8883), `PubSubClient` publish/subscribe tương ứng (**`main.cpp`**).

2.4 Xử lý dữ liệu bằng Node-RED

Node-RED là công cụ lập trình flow-based; dữ liệu từ ESP32 được mô tả chi tiết trong `node-red.json`. Message MQTT từ topic `iot/farm/env` vào node **mqtt in** rồi phân nhánh song song.

Nhánh thứ nhất (**Gauge nhiệt độ**): message được chuyển qua node function trích giá trị `msg.payload.temperature`, chuyển thành số thực, rồi truyền tới node **ui_gauge** để hiển thị kim đồng hồ đo nhiệt độ với ba vùng màu (xanh: bình thường, vàng: trung gian, đỏ: nguy hiểm).

Nhánh thứ hai (**Gauge độ ẩm không khí**): tương tự, trích giá trị `msg.payload.humidity`, truyền tới node **ui_gauge** (dạng donut) hiển thị độ ẩm không khí trong nhóm **Không khí**.

Nhánh thứ ba (**Gauge độ ẩm đất**): trích giá trị `msg.payload.soilMoisture` từ potentiometer (đã quy đổi phần trăm), truyền tới node **ui_gauge** dạng donut với ba vùng màu (đỏ: khô, vàng: trung bình, xanh: ẩm).

Nhánh thứ tư (**Gauge cường độ ánh sáng**): trích giá trị `msg.payload.lightLux`, truyền tới node **ui_gauge** hiển thị cường độ ánh sáng với cảnh báo khi < 1000 lux.

Nhánh thứ năm (**Chart nhiệt độ**): trích giá trị nhiệt độ, gán topic `Nhiệt độ °C`, truyền tới node **ui_chart** (line chart) để vẽ đồ thị theo thời gian thực, tự động xóa điểm cũ sau 1 giờ (`removeOlderUnit: 3600`).

Nhánh thứ sáu (**Chart độ ẩm KK**): trích giá trị độ ẩm không khí, gán topic `Độ ẩm KK \%`, truyền tới node **ui_chart** để vẽ đồ thị.

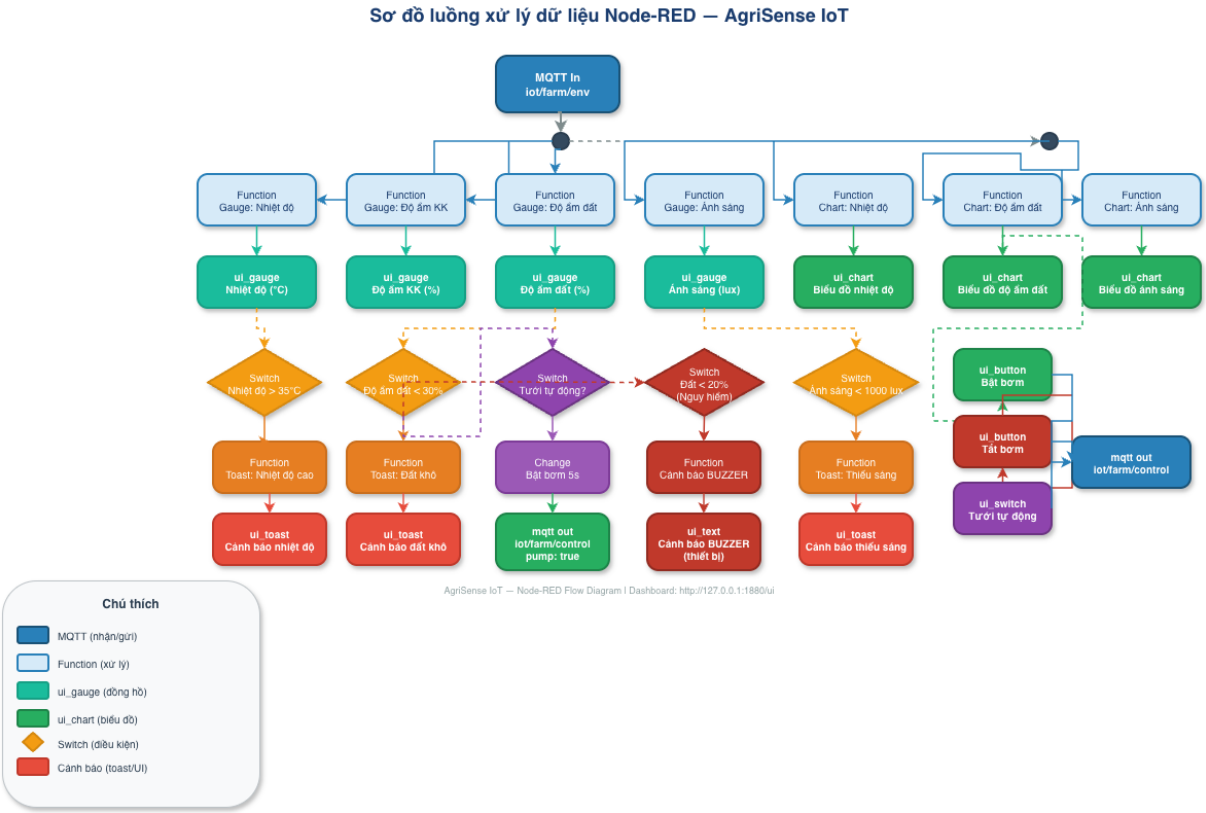
Nhánh thứ bảy (**Chart độ ẩm đất**): trích giá trị độ ẩm đất (đã chuyển %), gán topic `Độ ẩm đất \%`, truyền tới node **ui_chart** để vẽ đồ thị.

Nhánh thứ tám (**Cảnh báo nhiệt độ cao**): switch node kiểm tra `payload.temperature > 35`. Nếu đúng, function node tạo message cảnh báo, truyền tới node **ui_toast** hiển thị thông báo ở góc dưới bên trái Dashboard trong 8 giây.

Nhánh thứ chín (**Cảnh báo đất khô**): switch node kiểm tra `payload.soilMoisture < 30`. Nếu đúng, function node tạo toast thông báo đất khô kèm giá trị phần trăm. Ngưỡng nguy hiểm < 20% sẽ phát cảnh báo đỏ.

Nhánh thứ mười (**Tưới tự động**): switch node kiểm tra độ ẩm đất < 30% đồng thời

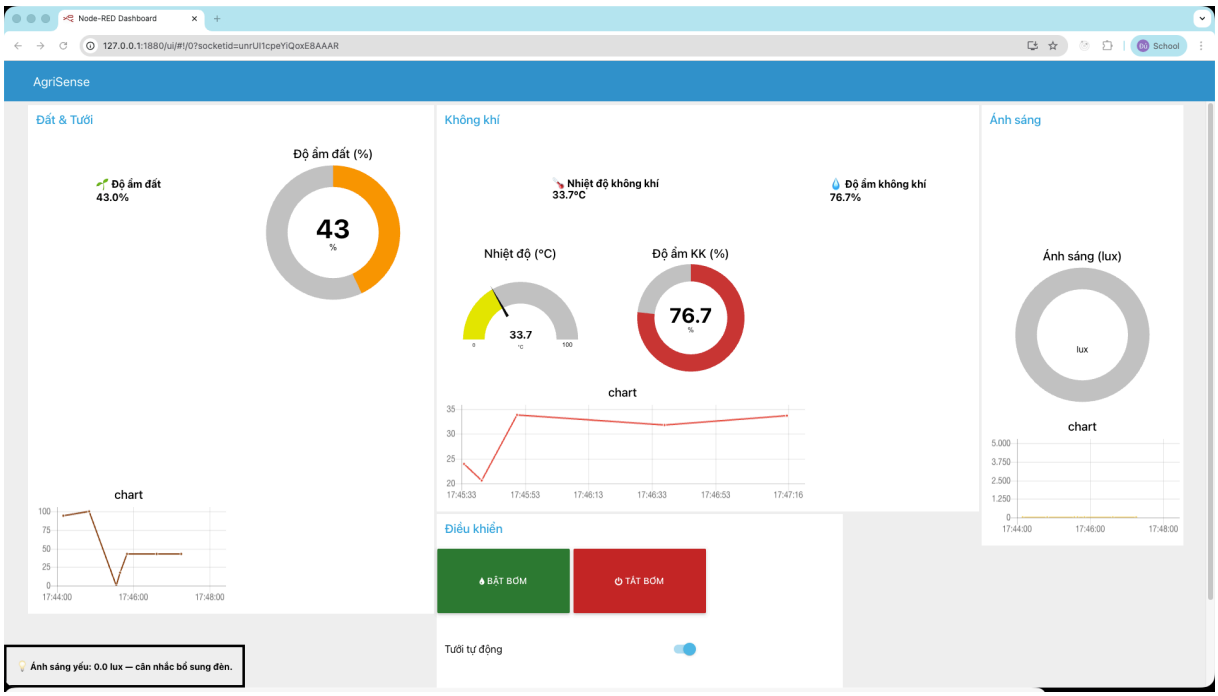
`flow.auto == true` (switch *Tưới tự động* trên Dashboard gán biến flow tên `auto`). Nếu thỏa điều kiện, change node tạo cấu trúc JSON `\{"pump":true,"duration":5\}` và publish thông qua node **mqtt out** đến topic `iot/farm/control`.



Hình 4: Sơ đồ luồng xử lý dữ liệu Node-RED — AgriSense IoT: MQTT `iot/farm/env` phân nhánh gauge, chart, cảnh báo và điều khiển `iot/farm/control`

2.5 Hiển thị dữ liệu trên Dashboard

Node-RED Dashboard cung cấp các widget: gauge (nhiệt độ, độ ẩm KK, độ ẩm đất, cường độ ánh sáng), chart đường theo thời gian thực, text trạng thái, nút **Máy bơm**, switch **Tưới tự động**, toast cảnh báo. Giao diện truy cập tại `http://localhost:1880/ui` (hoặc `http://127.0.0.1:1880/ui`).



Hình 5: Giao diện Node-RED Dashboard AgriSense: giám sát đất và tưới, không khí, ánh sáng; nút bật/tắt bơm và tưới tự động; ví dụ toast cảnh báo ánh sáng yếu (`127.0.0.1:1880/ui`)

2.6 Cảnh báo môi trường

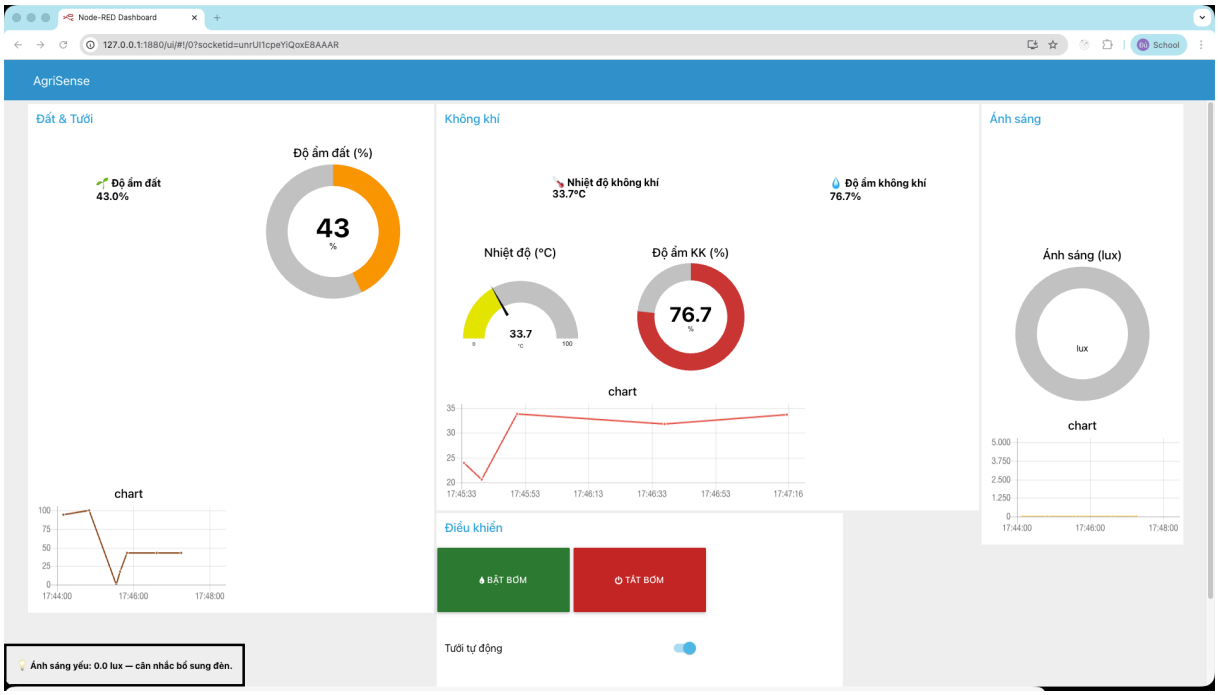
Trên Dashboard: toast khi `temperature > 35`°C, khi `soilMoisture < 30\%` (đất khô), khi `soilMoisture < 20\%` (nguy hiểm), và khi `lightLux < 1000` (thiếu sáng) (**`node-red.json`**)
Trên ESP32: buzzer phát bíp 2 lần khi `soilMoisture < 30\%`, phát bíp 5 lần khi `soilMoisture < 20\%` trong `loop()` (**`main.cpp`**).

Hình 6: Giao diện cảnh báo môi trường trên Node-RED Dashboard (toast nhiệt độ, độ ẩm đất, ánh sáng)

2.7 Điều khiển máy bơm từ xa qua MQTT

Hệ thống cung cấp khả năng điều khiển thiết bị chấp hành thông qua các bản tin định hướng từ máy chủ. Cụ thể, thiết bị ESP32 thực hiện subscribe vào topic điều khiển `iot/farm/control` và topic cấu hình `iot/farm/config` để chờ nhận lệnh. Khi nhận được một gói tin JSON chứa chỉ thị kích hoạt máy bơm, bộ vi xử lý sẽ tiến hành parse dữ liệu và thực thi việc bật Relay điều khiển máy bơm. Để đảm bảo an toàn và tiết kiệm

tài nguyên, hệ thống tích hợp cơ chế tự động ngắt sau một khoảng thời gian vận hành dự kiến (từ JSON `duration` hoặc mặc định 10 giây). Quy trình này giúp duy trì sự đồng bộ giữa trạng thái hiển thị trên Dashboard và hoạt động thực tế của thiết bị, đồng thời bảo vệ hệ thống khỏi các lỗi vận hành kéo dài không mong muốn.



Hình 7: Giao diện Dashboard AgriSense: khối *Điều khiển* với nút Bật/Tắt bơm và công tắc Tưới tự động (`127.0.0.1:1880/ui`)

2.8 Duy trì kết nối và đồng bộ trạng thái bơm

Hệ thống được thiết kế để tự động giám sát và duy trì tính ổn định của các kết nối mạng. Trong vòng lặp chính của chương trình, vi điều khiển liên tục kiểm tra trạng thái của Wi-Fi và giao thức MQTT. Nếu phát hiện sự cố ngắt kết nối, quy trình tái lập liên kết sẽ được thực hiện định kỳ cho đến khi thông tin giao tiếp được khôi phục. Đồng thời, cơ chế giám sát thời gian vận hành máy bơm cũng được thực thi song song để đảm bảo thiết bị tự động ngắt sau một khoảng thời gian quy định, giữ cho trạng thái thực tế luôn khớp với lệnh điều khiển đã gửi.

2.9 Tự động tưới theo ngưỡng độ ẩm đất

Hệ thống tích hợp logic điều khiển tự động dựa trên sự kết hợp giữa các thông số cảm biến thực tế và thiết lập từ người dùng. Khi chế độ tưới tự động được kích hoạt trên Dashboard, hệ thống sẽ liên tục đánh giá giá trị độ ẩm đất đo được. Nếu độ ẩm giảm xuống dưới ngưỡng 30% (đất khô), Node-RED hoặc ESP32 sẽ tự động tạo và publish các payload điều khiển qua giao thức MQTT để kích hoạt máy bơm nước tưới bù độ ẩm, đảm bảo duy trì môi trường lý tưởng cho cây trồng. Ngưỡng nguy hiểm 20% sẽ kích hoạt cảnh báo khẩn cấp với buzzer phát 5 lần liên tục.

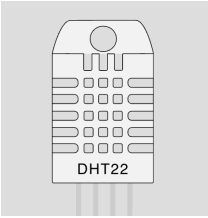
CHƯƠNG 3: THIẾT KẾ MẠCH IoT

3.1 Thành phần phần cứng

Hệ thống sử dụng các linh kiện điện tử cơ bản và phổ biến, được lựa chọn nhằm tối ưu chi phí nhưng vẫn đảm bảo độ tin cậy trong việc thu thập dữ liệu và cảnh báo. Các thành phần chính bao gồm:

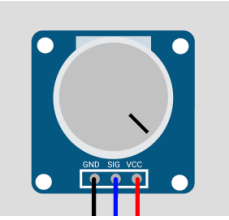
- **Vi điều khiển trung tâm:** ESP32 DevKit C v4 đóng vai trò bộ não xử lý, kết nối mạng Wi-Fi và giao tiếp MQTT.
- **Cảm biến nhiệt độ và độ ẩm không khí:** Cảm biến DHT22 đo nhiệt độ và độ ẩm không khí với sai số thấp.
- **Cảm biến cường độ ánh sáng:** Cảm biến BH1750 giao tiếp I2C đo cường độ ánh sáng (lux).
- **Cảm biến đo lường (mô phỏng):** Biến trở (Potentiometer) được sử dụng để giả lập tín hiệu điện áp tương tự (ADC) của cảm biến đo độ ẩm đất trong thực tế.
- **Cơ cấu chấp hành và cảnh báo:** Module Relay điều khiển máy bơm nước. Còi loa (Buzzer) điều khiển bằng xung PWM nhằm phát âm thanh báo động cường độ cao khi đất quá khô.

Hình 8: Vi điều khiển ESP32

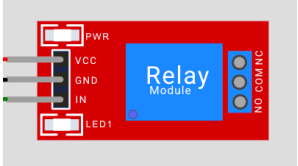


Hình 9: Cảm biến DHT22

Hình 10: Cảm biến BH1750



Hình 11: Biến trở



Hình 12: Module Relay

Hình 13: Còi Buzzer

3.2 Kết nối phần cứng (GPIO)

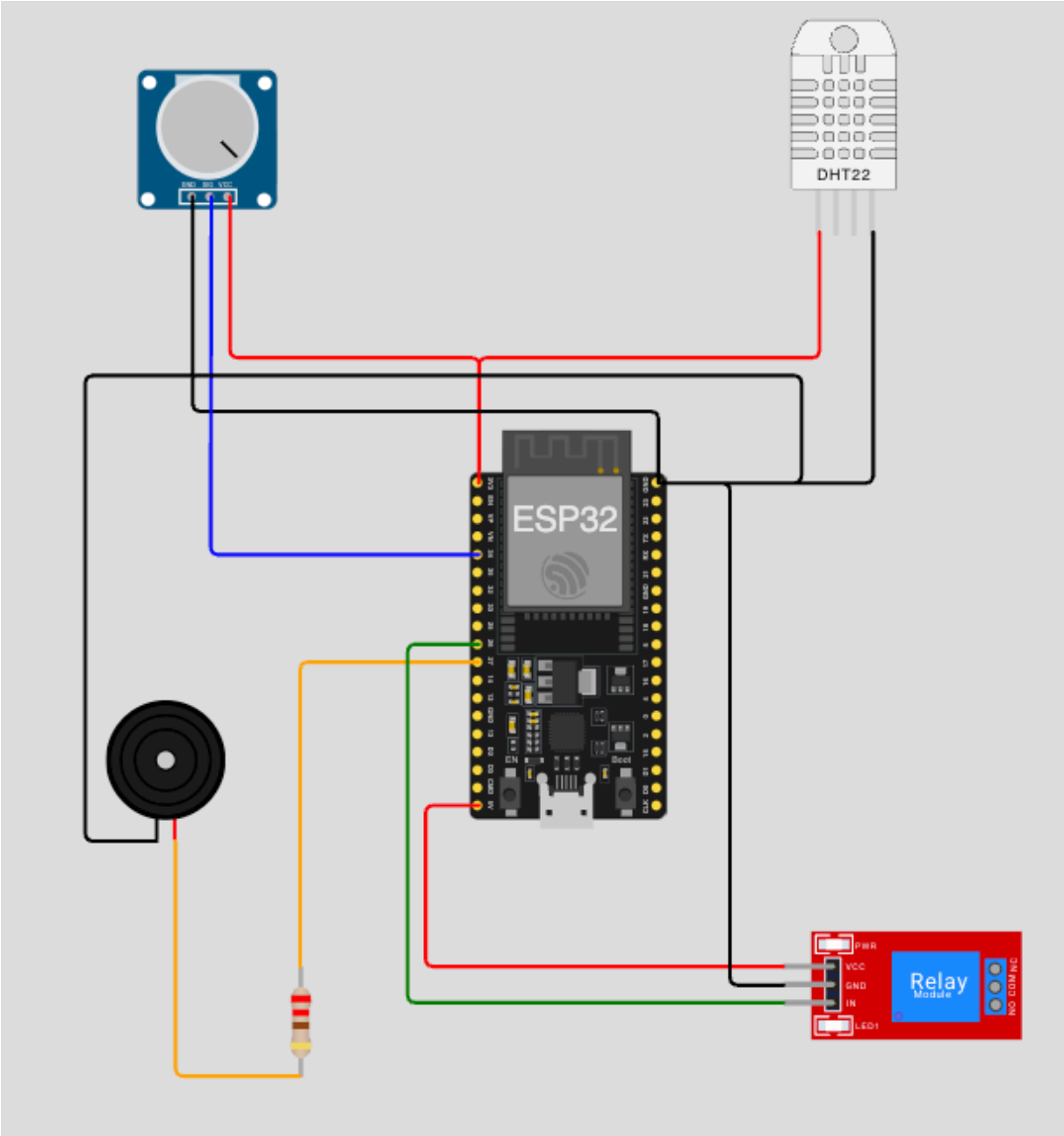
Bảng 1 tóm tắt nối dây khớp `main.cpp` và `diagram.json`.

Bảng 1: Bảng kết nối chân GPIO trên ESP32

Chân ESP32	**Nối đến**	**Chức năng**
3V3	DHT22 VCC, Potentiometer VCC, BH1750 VCC	Nguồn 3.3V cho cảm biến
5V	Relay VCC	Nguồn 5V cho Relay module
GND	DHT22 GND, Potentiometer GND, BH1750 GND, Buzzer GND, Relay GND	Chân nối đất chung
GPIO 4	DHT22 DATA	Bus dữ liệu 1-wire (nhiệt độ + độ ẩm KK)
GPIO 21	BH1750 SDA	Bus dữ liệu I2C (SDA)
GPIO 22	BH1750 SCL	Bus xung nhịp I2C (SCL)
GPIO 26	Relay IN	Điều khiển bật/tắt máy bơm
GPIO 27	Buzzer	Phát âm thanh cảnh báo khi đất khô
GPIO 34	Potentiometer SIG	Đọc giá trị độ ẩm đất (ADC, chỉ đọc)

3.2.1 Mạch vật lý — ảnh mô phỏng Wokwi (tương đương mạch thực tế đã nối)

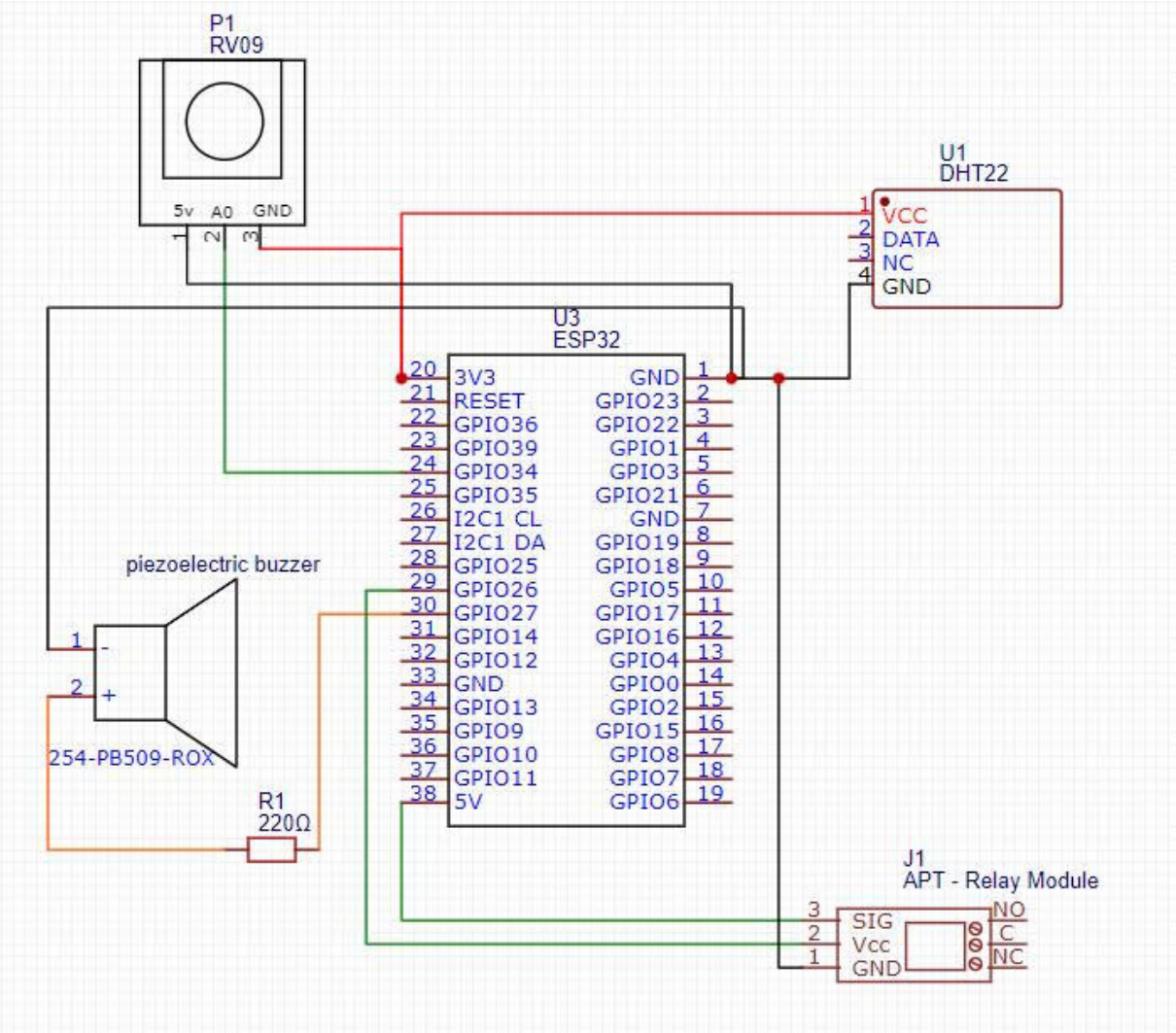
Hình 14.



Hình 14: Mạch mô phỏng Wokwi: ESP32 DevKit C v4 + DHT22 + BH1750 + Potentiometer + Relay + Buzzer (khớp `diagram.json`)

3.2.2 Sơ đồ mạch nguyên lý (EasyEDA)

Hình 15.



Hình 15: Sơ đồ nguyên lý phần cứng (EasyEDA): ESP32, DHT22, Potentiometer, Relay, Buzzer (cảm biến ánh sáng BH1750 được bổ sung trên mạch mô phỏng Wokwi)

3.3 Giải thích các linh kiện chính

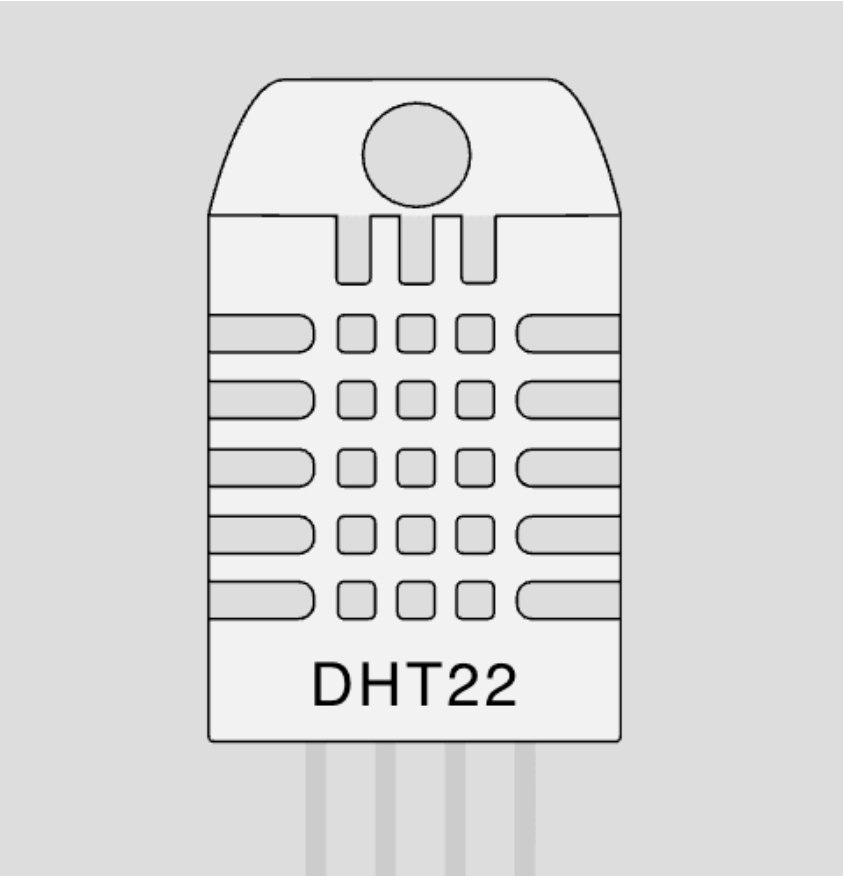
3.3.1 Vi điều khiển ESP32 DevKit C v4

ESP32 là một hệ thống trên chip (SoC) tích hợp Wi-Fi và Bluetooth, được xây dựng trên nền vi xử lý Xtensa LX6 dual-core hiệu năng cao. Với bộ chuyển đổi tín hiệu tương tự sang số (ADC) 12-bit có độ phân giải cao, ESP32 có khả năng đọc chính xác các tín hiệu điện áp từ cảm biến analog. Trong hệ thống này, ESP32 đóng vai trò trung tâm điều phối: thu thập dữ liệu từ cảm biến, xử lý tín hiệu và duy trì kết nối liên tục lên đám mây qua giao thức MQTT bảo mật.

Hình 16: Mạch vi điều khiển ESP32 DevKit C v4

3.3.2 Cảm biến nhiệt độ và độ ẩm không khí DHT22

DHT22 là cảm biến đo môi trường sử dụng giao tiếp số 1-wire, cho phép đo đồng thời nhiệt độ (độ chính xác ±0.5°C) và độ ẩm không khí (độ chính xác ±2–5%). So với thế hệ trước (DHT11), DHT22 cải thiện đáng kể về phạm vi đo và độ phân giải, phù hợp cho các ứng dụng nông nghiệp yêu cầu theo dõi môi trường chính xác.



Hình 17: Cảm biến DHT22 đo nhiệt độ và độ ẩm không khí

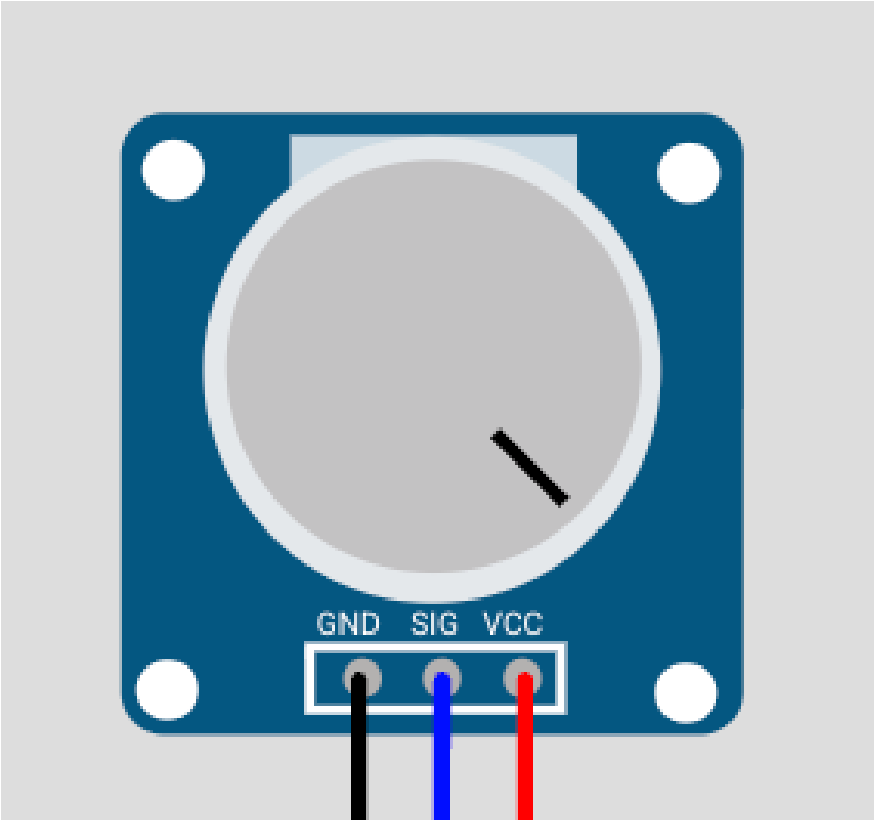
3.3.3 Cảm biến cường độ ánh sáng BH1750

BH1750 là cảm biến đo cường độ ánh sáng giao tiếp qua giao thức I2C, cho phép đo dải ánh sáng từ 1 lux đến 65535 lux với độ phân giải cao. Cảm biến này phù hợp để giám sát điều kiện ánh sáng cho quá trình quang hợp của cây trồng, với ngưỡng cảnh báo thiếu sáng < 1000 lux.

Hình 18: Cảm biến cường độ ánh sáng BH1750

3.3.4 Biến trở (mô phỏng cảm biến độ ẩm đất)

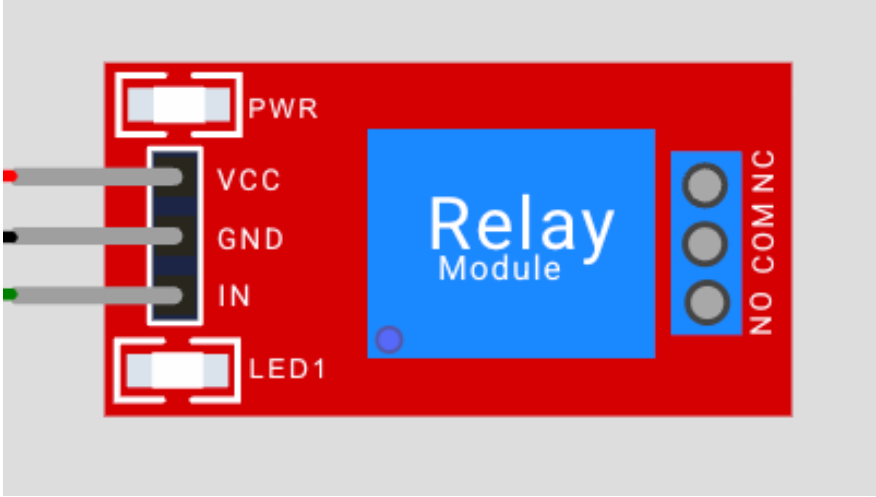
Trong phạm vi mô phỏng của đề tài, biến trở (Potentiometer) được sử dụng thay thế cho cảm biến đo độ ẩm đất chuyên dụng. Khi xoay trục biến trở, điện trở thay đổi làm thay đổi mức điện áp trên chân tín hiệu, từ đó mô phỏng dải giá trị từ đất khô hoàn toàn đến đất bão hòa nước. Kết quả đọc được qua bộ ADC sẽ được quy đổi sang phần trăm độ ẩm để hiển thị trực quan trên Dashboard.



Hình 19: Biến trở xoay dùng mô phỏng tín hiệu độ ẩm đất

3.3.5 Module Relay điều khiển máy bơm

Module Relay được sử dụng để điều khiển máy bơm nước. Relay hoạt động như một công tắc điện từ: khi nhận tín hiệu HIGH từ GPIO 26, tiếp điểm relay đóng lại, cho phép dòng điện chạy qua máy bơm. Khi nhận tín hiệu LOW, tiếp điểm mở ra, ngắt điện máy bơm. Đây là cách an toàn để điều khiển các thiết bị công suất cao từ vi điều khiển 3.3V.



Hình 20: Module Relay điều khiển máy bơm nước

3.3.6 Còi Buzzer cảnh báo

Còi sinh âm (Buzzer) được điều khiển bằng xung PWM từ GPIO 27, phát tín hiệu âm thanh cường độ cao khi hệ thống phát hiện độ ẩm đất xuống dưới ngưỡng cảnh báo. Buzzer phát bíp 2 lần khi đất khô (< 30%) và phát bíp 5 lần liên tục khi đất ở mức nguy hiểm (< 20%).

Hình 21: Còi Buzzer phát âm thanh cảnh báo

3.4 Luồng hoạt động tóm tắt theo chức năng

Toàn bộ hoạt động của hệ thống diễn ra theo một vòng lặp liên tục: vi điều khiển định kỳ thu thập dữ liệu từ các cảm biến, thực hiện kiểm tra ngưỡng để kích hoạt cảnh báo âm thanh khi cần, sau đó đóng gói kết quả thành bản tin JSON và gửi lên broker MQTT. Song song với đó, hệ thống luôn lắng nghe các lệnh điều khiển từ Dashboard để bật hoặc tắt máy bơm, đồng thời tự động ngắt bơm sau một khoảng thời gian chạy định sẵn nhằm tránh bơm hoạt động quá lâu. Chi tiết về sơ đồ kiến trúc và luồng dữ liệu được trình bày trong Chương 4.

Hình 22: Sơ đồ luồng hoạt động tóm tắt theo chức năng của hệ thống

CHƯƠNG 4: NGUYÊN LÝ HOẠT ĐỘNG

4.1 Sơ đồ luồng hệ thống (kiến trúc ba tầng)

Kiến trúc của hệ thống được xây dựng dựa trên mô hình ba tầng tiêu chuẩn của một ứng dụng IoT hiện đại. Tầng dưới cùng là tầng thiết bị, bao gồm vi điều khiển ESP32 kết nối trực tiếp với các cảm biến và cơ cấu chấp hành. Tầng chính giữa là tầng trung gian, sử dụng dịch vụ Broker của HiveMQ Cloud với giao thức bảo mật TLS để chuyển tiếp dữ liệu một cách an toàn. Trên cùng là tầng ứng dụng với Node-RED Dashboard, đóng vai trò là giao diện trung tâm để người dùng giám sát các thông số môi trường và phát lệnh điều khiển ngược trở lại thiết bị qua các chủ đề (topic) đã được định nghĩa thống nhất.

Hình 23: Sơ đồ kiến trúc tổng thể hệ thống giám sát và điều khiển môi trường trang trại thông minh

4.2 Luồng giám sát: cảm biến → MQTT → Dashboard

Luồng giám sát được thực hiện theo cơ chế định kỳ: cứ mỗi chu kỳ 5 giây, vi điều khiển đọc dữ liệu từ cảm biến nhiệt độ, độ ẩm không khí (DHT22), cường độ ánh sáng (BH1750) và tín hiệu ADC từ biến trở mô phỏng độ ẩm đất. Toàn bộ dữ liệu được đóng gói thành một bản tin JSON có cấu trúc thống nhất rồi gửi lên MQTT Broker (HiveMQ Cloud) qua kết nối bảo mật TLS. Broker sau đó phân phát bản tin tới Node-RED — nơi các luồng xử lý song song phân tích giá trị và cập nhật lên các widget tương ứng trên Dashboard: đồng hồ đo (gauge), biểu đồ theo thời gian thực (line chart) và thông báo cảnh báo (toast notification). Kiến trúc ba tầng của luồng này được minh họa tại Hình 23 (mục 4.1).

4.3 Luồng điều khiển: Dashboard → MQTT → ESP32 (Relay điều khiển bơm)

Người dùng bấm nút hoặc kích hoạt tưới tự động trên Dashboard; Node-RED publish JSON xuống `iot/farm/control`; `mqttCallback()` → `handleControlMessage()` bật Relay và ghi `pumpAutoOffTime`.
ESP32 thực hiện chu trình: khởi tạo Serial 115200, Wi-Fi, DHT, BH1750, GPIO, MQTT TLS, subscribe `iot/farm/control` và `iot/farm/config`. Vòng `loop()`: duy trì Wi-Fi/MQTT, `checkPumpTimeout()`, mỗi 5 s đọc cảm biến, buzzer nếu đất khô, `publishSensorData()` nếu broker sẵn sàng.

Hình 24: Lưu đồ thuật toán trên ESP32 (phần setup và loop)

Hình 25: Lưu đồ thuật toán trên ESP32 (phần đọc cảm biến, cảnh báo, MQTT)

4.4 Luồng đồng bộ trạng thái bơm và giới hạn thời gian chạy

Để tránh tình trạng máy bơm hoạt động không kiểm soát, hệ thống triển khai cơ chế giám sát thời gian chạy tự động. Ngay khi nhận lệnh kích hoạt bơm từ Dashboard, vi điều khiển ghi lại thời điểm bắt đầu vào biến `pumpAutoOffTime`. Khi thời gian vận hành đạt ngưỡng giới hạn (từ JSON `duration` hoặc mặc định 10 giây), hệ thống tự động ngắt Relay và cập nhật trạng thái bơm tương ứng — đảm bảo hành vi thực tế của thiết bị luôn đồng bộ với trạng thái hiển thị trên giao diện điều khiển.

4.5 Luồng tưới tự động theo ngưỡng độ ẩm đất (Node-RED)

Chế độ tưới tự động được kích hoạt dựa trên hai điều kiện đồng thời: giá trị độ ẩm đất đo được thấp hơn ngưỡng cảnh báo khô hạn, và người dùng đã bật chế độ tự động trên Dashboard. Khi cả hai điều kiện này được thỏa mãn, Node-RED tự động phát lệnh kích hoạt bơm mà không cần can thiệp thủ công. Cơ chế này hoàn toàn dựa trên giá trị cảm biến thực tế, không phụ thuộc vào lịch thời gian hay đồng hồ hệ thống.

4.6 Luồng duy trì kết nối Wi-Fi và MQTT

Tính ổn định của hệ thống IoT phụ thuộc lớn vào khả năng tự phục hồi kết nối mạng. Vi điều khiển liên tục kiểm tra trạng thái kết nối Wi-Fi trong mỗi vòng lặp; nếu phát hiện mất kết nối, hệ thống tự động thực hiện kết nối lại mà không làm gián đoạn các tác vụ khác. Tương tự, nếu phiên kết nối MQTT bị ngắt, hệ thống định kỳ thử kết nối lại tối đa mỗi 5 giây — đảm bảo luồng dữ liệu cảm biến và các lệnh điều khiển không bị gián đoạn lâu dài.

CHƯƠNG 5: CÀI ĐẶT HỆ THỐNG

5.1 Chuẩn bị công cụ và môi trường phát triển

Trước khi tiến hành cài đặt, người dùng cần chuẩn bị các phần mềm và môi trường nền tảng sau để đảm bảo hệ thống vận hành ổn định:

- **Visual Studio Code:** Trình soạn thảo mã nguồn chính, tải tại https://code.visualstudio.com/.
- **PlatformIO IDE:** Tiện ích mở rộng trong VS Code dùng để quản lý dự án và thư viện cho ESP32, thông tin tại https://platformio.org/.
- **Node.js:** Nền tảng thực thi JavaScript phía máy chủ, tải phiên bản LTS tại https://nodejs.org/.
- **Node-RED:** Công cụ lập trình luồng (flow-based), hướng dẫn cài đặt tại https://nodered.org/docs/getting-started/local.

5.2 Khởi tạo sơ đồ mạch mô phỏng và cấu hình phần cứng

Trong đề tài này, việc kiểm thử phần cứng được thực hiện thông qua nền tảng mô phỏng trực tuyến Wokwi (https://wokwi.com/). Quy trình thiết lập như sau:

1. Truy cập Wokwi và khởi tạo một dự án ESP32 mới.
2. Sao chép toàn bộ nội dung mã nguồn JSON tại **Phụ lục C (Sơ đồ mạch Wokwi)**.

3. Mở thẻ `diagram.json` trong trình soạn thảo của Wokwi và dán nội dung đã sao chép vào.

4. Hệ thống sẽ tự động khởi tạo các linh kiện (ESP32, DHT22, BH1750, Potentiometer, Relay, Buzzer) và các kết nối dây tương ứng theo đúng thiết kế của đề tài.

5.3 Triển khai và vận hành Firmware cho ESP32

5.3.1 Cài đặt mã nguồn và thư viện

Người dùng thực hiện sao chép toàn bộ mã nguồn chương trình tại **Phụ lục A (Mã nguồn ESP32)** và dán vào tệp `main.cpp` trong dự án PlatformIO hoặc thẻ `sketch.ino` trên Wokwi. Các thư viện như *DHT sensor library*, *PubSubClient*, *ArduinoJson* và `BH1750\_WE` sẽ được hệ thống tự động nhận diện và quản lý.

5.3.2 Cấu hình thông số hệ thống

Để thiết bị có thể kết nối với hạ tầng đám mây, cần cập nhật các thông số sau trong tệp mã nguồn:

- **Kết nối mạng:** Cập nhật tên và mật khẩu Wi-Fi. (Trên Wokwi mặc định dùng tên dự án và mật khẩu trống hoặc mạng giả lập).
- **Xác thực MQTT:** Cập nhật các thông tin định danh bao gồm địa chỉ máy chủ, tên đăng nhập và mật khẩu truy cập (Broker, Username, Password) sao cho khớp với tài khoản HiveMQ Cloud đã đăng ký.

5.3.3 Vận hành mô phỏng

Vì hệ thống sử dụng môi trường mô phỏng Wokwi, người dùng không cần kết nối thiết bị vật lý qua cổng USB. Chỉ cần nhấn nút **Start Simulation** để bắt đầu chạy chương trình. Toàn bộ quá trình kết nối và dữ liệu cảm biến sẽ được hiển thị trực tiếp qua cửa sổ *Serial Monitor* của trình giả lập.

5.4 Thiết lập hệ thống điều khiển và giao diện Node-RED

5.4.1 Lệnh cài đặt nền tảng

Mở cửa sổ dòng lệnh (Terminal) và thực hiện cài đặt Node-RED cùng giao diện Dashboard thông qua trình quản lý gói npm:

```
npm install -g node-red
npm install -g node-red-dashboard
```

Sau đó, gõ lệnh `node-red` để khởi động dịch vụ và truy cập vào địa trị http://127.0.0.1:1880.

5.4.2 Triển khai luồng xử lý và giao diện Dashboard

Để thiết lập logic điều khiển và giao diện, người dùng thực hiện:

1. Sao chép mã nguồn JSON tại **Phụ lục B (Mã nguồn Node-RED Flow)**.
2. Tại giao diện Node-RED, chọn **Menu** → **Import** và dán nội dung vào cửa sổ hiện ra.
3. Nhấn **Import** để hệ thống tự động tạo ra các luồng xử lý và giao diện Dashboard.
4. Cập nhật cấu hình node **mqtt-broker** để thiết lập thông tin đăng nhập HiveMQ Cloud tương tự như phía ESP32.

5.5 Vận hành và kiểm thử tổng thể

Sau khi hoàn tất cài đặt, người dùng truy cập Dashboard tại http://127.0.0.1:1880/ui. Hệ thống hoạt động đúng khi các đồng hồ đo hiển thị giá trị nhiệt độ, độ ẩm không khí, độ ẩm đất và cường độ ánh sáng từ Wokwi, và người dùng có thể thực hiện bật/tắt máy bơm từ xa thông qua các nút nhấn trên giao diện.

Hình 26: Giao diện editor Node-RED với flow đầy đủ

Hình 27: Giao diện Dashboard Node-RED (phần 1/2): độ ẩm không khí, nhiệt độ, cường độ ánh sáng, điều khiển máy bơm và tưới tự động

Hình 28: Giao diện Dashboard Node-RED (phần 2/2): nhiệt độ, điều khiển, độ ẩm đất (gauge, biểu đồ line, trạng thái đất và cảnh báo)

CHƯƠNG 6: KẾT QUẢ THỰC HIỆN

6.1 Kết quả mô phỏng phần cứng trên Wokwi

Mạch điện của hệ thống được thực hiện mô phỏng đầy đủ trên nền tảng Wokwi. Hình 29 cho thấy sự kết nối chính xác giữa ESP32 và các linh kiện như cảm biến DHT22, BH1750, biến trở mô phỏng độ ẩm đất, còi báo động Buzzer và Relay điều khiển máy bơm.

Hình 29: Mô phỏng sơ đồ kết nối phần cứng thực tế trên môi trường Wokwi

6.2 Kết quả truyền nhận dữ liệu qua Serial Monitor

Trong quá trình vận hành, vi điều khiển ESP32 liên tục gửi các thông tin trạng thái hoạt động lên cửa sổ Serial Monitor. Các thông tin bao gồm quá trình kết nối Wi-Fi, trạng thái thiết lập kết nối bảo mật TLS với HiveMQ Cloud và dữ liệu cảm biến thu thập được trước khi đóng gói JSON. Đoạn mã sau trích từ **main.cpp** — đúng các lệnh Serial tạo nên log tương ứng trên màn hình (Hình 30).

```
-ảếấậấậấớ-
ỏđộẩđấỨỮẢĐấ→ầể→ầế→ảẢẾẠạớậởạếđộướựđộờđểựấớịấĐỀỂƠĐỌẢẾưởạ→ảịặđịĐỌĐỘẨĐẤ→ỬỆĐỀỂỪệậấớếậớ
ậẾỔ
void setupWiFi() {
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(100);
    Serial.print(".");
  }

  Serial.println();
  Serial.print("WiFi connected! IP: ");
```

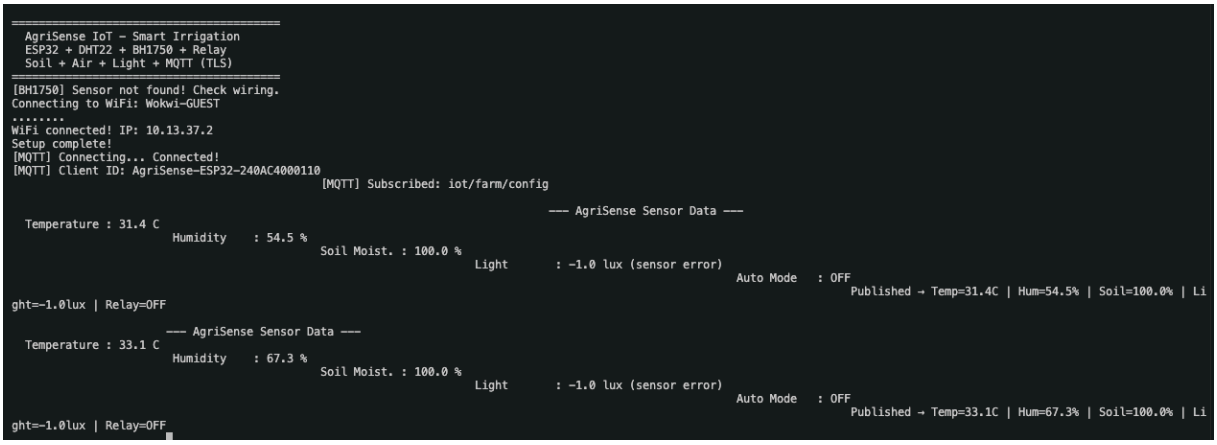
```
  Serial.println(WiFi.localIP());
}

//
    ==============================================================
// ẾKT ỐNI MQTT
//
    ==============================================================
void reconnectMQTT() {
  Serial.print("[MQTT] Connecting...");
  // Client ID ốc định ựda trên MAC - đảm ảbo duy ấnht trên
   HiveMQ
  String clientId = "AgriSense-ESP32-";
  clientId += WiFi.macAddress();
  clientId.replace(":", "");

  if (mqttClient.connect(clientId.c_str(), MQTT_USERNAME,
   MQTT_PASSWORD)) {
    Serial.println(" Connected!");
    mqttClient.subscribe(TOPIC_CONTROL);
    Serial.printf("[MQTT] Client ID: %s\n", clientId.c_str());
    mqttClient.subscribe(TOPIC_CONFIG);
    Serial.printf("[MQTT] Subscribed: %s\n", TOPIC_CONFIG);
  } else {
    Serial.printf(" Failed, state: %d\n", mqttClient.state());
  }
}ŨỆ→ĐỤỘẢẾệđộđộẩĐộẩđấỬẠ
bool initBH1750() {
  Wire.begin(I2C_SDA, I2C_SCL);
  delay(100); // Đợi I2C ổn định
  if (!bh1750.init()) {
    Serial.println("[BH1750] Sensor not found! Check wiring.")
   ;
    return false;
  }
  bh1750.setMode(CHM);
  Serial.println("[BH1750] Initialized OK (mode: High-Res)");
  return true;
}
```

```
//
    ==============================================================
// SETUP
//
    ==============================================================
void setup() {
  Serial.begin(115200);
  Serial.println("\n========================================")
   ;
  Serial.println("  AgriSense IoT - Smart Irrigation");
  Serial.println("  ESP32 + DHT22 + BH1750 + Relay");
  Serial.println("  Soil + Air + Light + MQTT (TLS)");
  Serial.println("========================================");

  // --- GPIO ---
  pinMode(PUMP_RELAY_PIN, OUTPUT);
  digitalWrite(PUMP_RELAY_PIN, LOW); // Relay OFF ban đầu

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  pinMode(SOIL_PIN, INPUT);

  // --- DHT22 ---
  dht.begin();

  // --- BH1750 (I2C) ---
  lightSensorOK = initBH1750();

  // --- TLS ---
  espClient.setInsecure();

  // --- Wi-Fi ---
  setupWiFi();

  // --- MQTT ---
  mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
  mqttClient.setCallback(mqttCallback);
```

```
  Serial.println("Setup complete!");
}ĂỆỰĐỘẮƠạõ-ƯỨỰĐỘạõĐọđộẩđẩụếđợỉậđếơđẩđẩệậạờ-ậơđịỳ
    Serial.println("\n--- AgriSense Sensor Data ---");
    Serial.printf("  Temperature : %.1f C\n", temperature);
    Serial.printf("  Humidity    : %.1f %%\n", humidity);
    Serial.printf("  Soil Moist. : %.1f %%\n", soilMoisture);
    Serial.printf("  Light       : %.1f lux %s\n", lightLux,
                  lightLux < 0 ? "(sensor error)" : "");
    Serial.printf("  Auto Mode   : %s\n", autoIrrigationEnabled ? "ON" : "OFF");

    // === ẢCNH BÁO ĐỘ ẨM ĐẤT ===
    if (soilMoisture < SOIL_CRITICAL_THRESHOLD) {
      Serial.println("[ALERT] Soil CRITICAL - buzzer!");
      beepAlert(5, 300, 200);
    } else if (soilMoisture < SOIL_DRY_THRESHOLD) {
      Serial.println("[WARN] Soil is DRY!");
      beepAlert(2, 150, 150);
    }

    // === ẢCNH BÁO ÁNH SÁNG (ỉch log, buzzer do Dashboard ửx lý) ===
    if (lightSensorOK && lightLux >= 0 && lightLux < LIGHT_LOW_THRESHOLD) {
      Serial.printf("[WARN] Low light: %.0f lux < %.0f\n", lightLux,
                    LIGHT_LOW_THRESHOLD);
    }

    // === ỬGI MQTT ===
    if (mqttClient.connected()) {
      publishSensorData(temperature, humidity, soilMoisture, lightLux);

      // Đồng ộb relay ạtrng thái ớvi MQTT
      digitalWrite(PUMP_RELAY_PIN, pumpRelayState ? HIGH : LOW);
    } else {
      Serial.println("[MQTT] Not connected, skipping publish...");
```

```
}
```

Listing 1: Các hàm và đoạn `loop` in log lên Serial Monitor — trích **`src/main.cpp`**



Hình 30: Dữ liệu log hoạt động và trạng thái kết nối MQTT trên Serial Monitor (minh họa tương ứng Listing 1)

6.3 Thiết lập luồng xử lý dữ liệu trên Node-RED

Mọi chương trình xử lý phía máy chủ được tổ chức thành các luồng (flow) trực quan trên Node-RED. Hình 31 thể hiện toàn bộ các node chức năng từ việc tiếp nhận dữ liệu MQTT, giải mã JSON đến việc phân luồng hiển thị và logic điều khiển máy bơm tự động.

Hình 31: Sơ đồ luồng xử lý dữ liệu (Flow) đầy đủ trên trình biên tập Node-RED

6.4 Giao diện giám sát và điều khiển Dashboard

Kết quả quan trọng nhất của hệ thống là giao diện tương tác người dùng hiện đại và trực quan. Hệ thống Dashboard cung cấp đầy đủ các thông tin cần thiết về môi trường theo thời gian thực (Hình 32 và Hình 33).

Hình 32: Giao diện Dashboard (Phần 1): Hiển thị đồng hồ đo và các nút điều khiển trung tâm

Hình 33: Giao diện Dashboard (Phần 2): Biểu đồ lịch sử dữ liệu và các thông tin trạng thái đất

6.5 Phân tích luồng hoạt động thực tế

Hệ thống được thiết kế để xử lý các tình huống môi trường thực tế thông qua các cơ chế phản hồi tự động và cảnh báo trực quan (Hình 34).

Cơ chế cảnh báo: Khi các thông số vượt ngưỡng an toàn, hệ thống sẽ tự động kích hoạt các chỉ báo:

- **Nhiệt độ:** Khi nhiệt độ > 35°C, Dashboard hiển thị toast cảnh báo nhiệt độ cao.
- **Độ ẩm đất:** Khi độ ẩm đất < 30%, toast cảnh báo đất khô sẽ xuất hiện và còi Buzzer trên ESP32 phát âm thanh báo động. Đặc biệt, khi độ ẩm giảm sâu (< 20%), Dashboard sẽ hiển thị cảnh báo mức đỏ cực kỳ khô hạn và buzzer phát 5 lần liên tục.
- **Cường độ ánh sáng:** Khi ánh sáng < 1000 lux, hệ thống cảnh báo thiếu sáng ảnh hưởng đến quang hợp của cây.

Hình 34: Minh họa các thông báo cảnh báo và biểu đồ trạng thái trên giao diện Dashboard

Cơ chế điều khiển: Người dùng có thể thực hiện publish lệnh điều khiển thông qua nút nhấn **Máy bơm** hoặc kích hoạt chế độ **Tưới tự động** để hệ thống tự xử lý dựa trên ngưỡng độ ẩm đất.

Kết quả mô phỏng: Môi trường Wokwi phản hồi chính xác các lệnh MQTT gửi đến; Relay điều khiển máy bơm bật/tắt đúng theo yêu cầu và còi báo động hoạt động đồng bộ với trạng thái cảm biến.

Bảng 2: Tổng hợp các chức năng đã triển khai

Chức năng	**Trạng thái**	**Ghi chú**
Đọc DHT22 mỗi 5 giây	Hoàn thành	GPIO 4, fallback random khi lỗi
Đọc BH1750 mỗi 5 giây	Hoàn thành	GPIO 21-22 (I2C)
Đọc độ ẩm đất (ADC)	Hoàn thành	GPIO 34, potentiometer trên Wokwi
Publish MQTT (TLS)	Hoàn thành	Topic iot/farm/env, cổng 8883
Subscribe MQTT (TLS)	Hoàn thành	Topic iot/farm/control, iot/farm/config
Hiển thị Gauge nhiệt độ	Hoàn thành	3 vùng màu, min=0, max=100
Hiển thị Gauge độ ẩm KK	Hoàn thành	dạng donut, min=0, max=100
Hiển thị Gauge độ ẩm đất	Hoàn thành	dạng donut, min=0, max=100, 3 vùng màu
Hiển thị Gauge cường độ sáng	Hoàn thành	min=0, max=65535 lux
Biểu đồ line thời gian thực	Hoàn thành	dot=true, removeOlderUnit=3600
Toast cảnh báo nhiệt độ cao	Hoàn thành	Ngưỡng > 35°C, 8 giây
Toast cảnh báo đất khô	Hoàn thành	Ngưỡng < 30%, 8 giây

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Các công việc đã hoàn thành

Qua quá trình triển khai, đề tài đạt các mục tiêu gắn với mã nguồn đính kèm:

Thứ nhất, hệ thống IoT ba tầng hoàn chỉnh: ESP32 + DHT22 + BH1750 + Potentiometer + Relay + Buzzer; HiveMQ Cloud (TLS); Node-RED Dashboard.

Thứ hai, thu thập và truyền bốn thông số mỗi 5 giây qua MQTT (`iot/farm/env`), JSON chuẩn: nhiệt độ, độ ẩm không khí, độ ẩm đất, cường độ ánh sáng.

Thứ ba, Dashboard hiển thị gauge, biểu đồ, toast cảnh báo; điều khiển bơm và tưới tự động qua `iot/farm/control`.

Thứ tư, mô phỏng Wokwi và sơ đồ EasyEDA minh họa thiết kế.

Hệ thống có thể ứng dụng cho nhà kính, ban công, sân thượng.

Ưu điểm: Chi phí thấp; công cụ mã nguồn mở; MQTT và TLS phù hợp IoT; tách luồng Node-RED dễ bảo trì; reconnect MQTT không chặn `loop()`; fallback khi DHT22 trả NaN; cảnh báo buzzer đa cấp (2 lần khi khô, 5 lần khi nguy hiểm).

7.2 Công việc chưa thực hiện và hạn chế

Chưa triển khai thực tế ngoài mô phỏng Wokwi; chưa dùng cảm biến độ ẩm đất chuyên dụng (dùng potentiometer); `setInsecure()` chỉ phù hợp thử nghiệm; chưa lưu lịch sử ngoài biểu đồ Dashboard.

7.3 Hướng phát triển tiếp theo

Thứ nhất, thay potentiometer bằng cảm biến độ ẩm đất capacitive thực tế.

Thứ hai, tích hợp relay và máy bơm nước thực tế cho trang trại.

Thứ ba, xác thực TLS đầy đủ (CA), bỏ `setInsecure()` khi triển khai thực tế.

Thứ tư, InfluxDB/MySQL cho lịch sử và báo cáo.

Thứ năm, ứng dụng di động thay hoặc bổ sung Dashboard.

Thứ sáu, OTA để cập nhật firmware từ xa.

TÀI LIỆU THAM KHẢO

Tài liệu

[1] Espressif Systems. *ESP32 Series Datasheet*. Truy cập: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

[2] Node-RED. *Node-RED Documentation*. IBM & others. Truy cập: https://nodered.org/docs/.

[3] HiveMQ. *HiveMQ Cloud*. Truy cập: https://www.hivemq.com/cloud/.

[4] Seeed Studio. *Capacitive Soil Moisture Sensor*. Truy cập: https://wiki.seeedstudio.com/Capacitive_Soil_Moisture_Sensor/.

[5] Adafruit. *DHT sensor library for Arduino*. Truy cập: https://github.com/adafruit/DHT-sensor-library.

[6] Nick O'Leary. *PubSubClient for Arduino*. Truy cập: https://github.com/knolleary/pubsubclient.

[7] Node-RED. *node-red-dashboard*. Truy cập: https://flows.nodered.org/node/node-red-dashboard.

CÁC PHỤ LỤC

PHỤ LỤC A. Mã nguồn chương trình ESP32

Lưu ý: Trước khi nạp firmware, cần cập nhật các thông tin sau trong code: `WIFI_SSID` và `WIFI_PASSWORD` (tên và mật khẩu Wi-Fi); `MQTT_USERNAME` và `MQTT_PASSWORD` (tài khoản HiveMQ Cloud). Thông tin xác thực MQTT đã được ẩn trong tài liệu này vì lý do bảo mật.

```
#include <Arduino.h>
#include <ArduinoJson.h>
#include <DHT.h>
#include <PubSubClient.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>

#include <BH1750_WE.h>
#include <Wire.h>

#ifdef WOKWI
const char *WIFI_SSID = "Wokwi-GUEST";
const char *WIFI_PASSWORD = "";
#else
const char *WIFI_SSID = "";
const char *WIFI_PASSWORD = "";
#endif

const char *MQTT_BROKER = "";
const int MQTT_PORT = 8883;
const char *MQTT_CLIENT_ID = "esp32-agrisense-01";
const char *MQTT_USERNAME = "";
const char *MQTT_PASSWORD = "";

const char *TOPIC_ENV = "iot/farm/env";
const char *TOPIC_CONTROL = "iot/farm/control";
const char *TOPIC_CONFIG = "iot/farm/config";

#define DHT_PIN 4
#define DHT_TYPE DHT22
```

```
#define PUMP_RELAY_PIN 26
#define BUZZER_PIN 27
#define SOIL_PIN 34

#define I2C_SDA 21
#define I2C_SCL 22

#define SOIL_DRY_THRESHOLD 30
#define SOIL_CRITICAL_THRESHOLD 20
#define LIGHT_LOW_THRESHOLD 1000.0

DHT dht(DHT_PIN, DHT_TYPE);
BH1750_WE bh1750;
WiFiClientSecure espClient;
PubSubClient mqttClient(espClient);

unsigned long lastPublish = 0;
unsigned long lastMQTTRetry = 0;
const unsigned long PUBLISH_INTERVAL = 5000;
const unsigned long MQTT_RETRY_MS = 5000;

bool pumpRelayState = false;
bool lightSensorOK = false;
bool autoIrrigationEnabled = false;
unsigned long pumpAutoOffTime = 0;

void beepAlert(int times, int onMs, int offMs) {
  for (int i = 0; i < times; i++) {
    digitalWrite(BUZZER_PIN, HIGH);
    delay(onMs);
    digitalWrite(BUZZER_PIN, LOW);
    if (i < times - 1)
      delay(offMs);
  }
}

void setPumpRelay(bool on) {
  pumpRelayState = on;
  digitalWrite(PUMP_RELAY_PIN, on ? HIGH : LOW);
  Serial.printf("[RELAY] Pump %s\n", on ? "ON" : "OFF");
}
```

```

float readLightLux() {
  if (!lightSensorOK) {
    return -1.0f;
  }

  float lux = bh1750.getLux();
  if (lux < 0)
    return -1.0f;
  return lux;
}

float readSoilMoisture() {
  int raw = analogRead(SOIL_PIN);
  float moisture = map(constrain(raw, 0, 4095), 0, 4095, 0, 100);
  return moisture;
}

void handleControlMessage(const char *payload) {
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, payload);
  if (err) {
    Serial.printf("[CONTROL] JSON parse error: %s\n", err.c_str());
    return;
  }

  if (doc.containsKey("pump")) {
    bool pumpOn = doc["pump"].as<bool>();
    int duration = 0;
    if (doc.containsKey("duration")) {
      duration = doc["duration"].as<int>();
    }
    if (pumpOn && duration > 0) {
      pumpAutoOffTime = millis() + (unsigned long)duration * 1000UL;
      Serial.printf("[CONTROL] Pump ON (auto off in %ds)\n", duration);
    } else if (!pumpOn) {
      pumpAutoOffTime = 0;
```

```
    }
    setPumpRelay(pumpOn);
    Serial.printf("[CONTROL] Pump command: %s\n", pumpOn ? "ON" : "OFF");
  }

  if (doc.containsKey("auto")) {
    bool autoOn = doc["auto"].as<bool>();
    autoIrrigationEnabled = autoOn;
    Serial.printf("[CONTROL] Auto irrigation: %s\n", autoOn ? "ON" : "OFF");
  }
}

void mqttCallback(char *topic, byte *payload, unsigned int length) {
  Serial.printf("[MQTT] Topic: %s\n", topic);

  char msg[length + 1];
  memcpy(msg, payload, length);
  msg[length] = '\0';
  Serial.printf("[MQTT] Payload: %s\n", msg);

  if (strcmp(topic, TOPIC_CONTROL) == 0) {
    handleControlMessage(msg);
  } else if (strcmp(topic, TOPIC_CONFIG) == 0) {
    handleControlMessage(msg);
  }
}

void setupWiFi() {
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(100);
    Serial.print(".");
  }
```

```
  Serial.println();
  Serial.print("WiFi connected! IP: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  Serial.print("[MQTT] Connecting...");
  String clientId = "AgriSense-ESP32-";
  clientId += WiFi.macAddress();
  clientId.replace(":", "");

  if (mqttClient.connect(clientId.c_str(), MQTT_USERNAME,
   MQTT_PASSWORD)) {
    Serial.println(" Connected!");
    mqttClient.subscribe(TOPIC_CONTROL);
    Serial.printf("[MQTT] Client ID: %s\n", clientId.c_str());
    mqttClient.subscribe(TOPIC_CONFIG);
    Serial.printf("[MQTT] Subscribed: %s\n", TOPIC_CONFIG);
  } else {
    Serial.printf(" Failed, state: %d\n", mqttClient.state());
  }
}

void publishSensorData(float temp, float hum, float soil,
   float lux) {
  StaticJsonDocument<512> doc;
  doc["device"] = "AgriSense-01";
  doc["temperature"] = temp;
  doc["humidity"] = hum;
  doc["soilMoisture"] = soil;
  doc["lightLux"] = (lux >= 0) ? lux : 0;
  doc["relayStatus"] = pumpRelayState ? "ON" : "OFF";
  doc["timestamp"] = millis();

  char buffer[512];
  size_t n = serializeJson(doc, buffer);
  mqttClient.publish(TOPIC_ENV, buffer, n);

  Serial.printf("Published -> Temp=%.1fC | Hum=%.1f%% | Soil
   =%.1f%% | "
```

```
                  "Light=%.1flux | Relay=%s\n",
                  temp, hum, soil, lux, pumpRelayState ? "ON" : "OFF");
}

void readSensors(float &temp, float &hum, float &soil, float &lux) {
  temp = dht.readTemperature();
  hum = dht.readHumidity();

  if (isnan(temp)) {
    temp = random(200, 350) / 10.0f;
  }
  if (isnan(hum)) {
    hum = random(400, 800) / 10.0f;
  }

  soil = readSoilMoisture();
  lux = readLightLux();
}

bool initBH1750() {
  Wire.begin(I2C_SDA, I2C_SCL);
  delay(100);
  if (!bh1750.init()) {
    Serial.println("[BH1750] Sensor not found! Check wiring.");
    return false;
  }
  bh1750.setMode(CHM);
  Serial.println("[BH1750] Initialized OK (mode: High-Res)");
  return true;
}

void setup() {
  Serial.begin(115200);
  Serial.println("\n========================================");
  Serial.println("  AgriSense IoT - Smart Irrigation");
  Serial.println("  ESP32 + DHT22 + BH1750 + Relay");
  Serial.println("  Soil + Air + Light + MQTT (TLS)");
```

```
  Serial.println("========================================");

  pinMode(PUMP_RELAY_PIN, OUTPUT);
  digitalWrite(PUMP_RELAY_PIN, LOW);

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  pinMode(SOIL_PIN, INPUT);

  dht.begin();

  lightSensorOK = initBH1750();

  espClient.setInsecure();

  setupWiFi();

  mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
  mqttClient.setCallback(mqttCallback);

  Serial.println("Setup complete!");
}

void loop() {
  unsigned long now = millis();

  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("[WiFi] Lost! Reconnecting...");
    setupWiFi();
  }

  if (!mqttClient.connected()) {
    if (now - lastMQTTRetry >= MQTT_RETRY_MS) {
      lastMQTTRetry = now;
      reconnectMQTT();
    }
  }
  mqttClient.loop();

  if (pumpRelayState && pumpAutoOffTime > 0 && now >=
```

```
  pumpAutoOffTime) {
    Serial.println("[PUMP] Auto-off timer reached -- turning off pump");
    pumpAutoOffTime = 0;
    setPumpRelay(false);
  }

  if (autoIrrigationEnabled) {
    float currentSoil = readSoilMoisture();
    if (!pumpRelayState && currentSoil < SOIL_DRY_THRESHOLD && pumpAutoOffTime == 0) {
      Serial.printf("[AUTO-IRRIG] Soil=%%%.1f < %%%d -- starting auto pump (10s)\n",
                    currentSoil, SOIL_DRY_THRESHOLD);
      pumpAutoOffTime = now + 10000UL;
      setPumpRelay(true);
    }
  }

  if (now - lastPublish >= PUBLISH_INTERVAL) {
    lastPublish = now;

    float temperature, humidity, soilMoisture, lightLux;
    readSensors(temperature, humidity, soilMoisture, lightLux);

    Serial.println("\n--- AgriSense Sensor Data ---");
    Serial.printf("  Temperature : %.1f C\n", temperature);
    Serial.printf("  Humidity    : %.1f %%\n", humidity);
    Serial.printf("  Soil Moist. : %.1f %%\n", soilMoisture);
    Serial.printf("  Light       : %.1f lux %s\n", lightLux,
                  lightLux < 0 ? "(sensor error)" : "");
    Serial.printf("  Auto Mode   : %s\n", autoIrrigationEnabled ? "ON" : "OFF");

    if (soilMoisture < SOIL_CRITICAL_THRESHOLD) {
      Serial.println("[ALERT] Soil CRITICAL - buzzer!");
      beepAlert(5, 300, 200);
    } else if (soilMoisture < SOIL_DRY_THRESHOLD) {
      Serial.println("[WARN] Soil is DRY!");
      beepAlert(2, 150, 150);
```

```
    }

    if (lightSensorOK && lightLux >= 0 && lightLux < LIGHT_LOW_THRESHOLD) {
      Serial.printf("[WARN] Low light: %.0f lux < %.0f\n", lightLux,
                    LIGHT_LOW_THRESHOLD);
    }

    if (mqttClient.connected()) {
      publishSensorData(temperature, humidity, soilMoisture, lightLux);

      digitalWrite(PUMP_RELAY_PIN, pumpRelayState ? HIGH : LOW);
    } else {
      Serial.println("[MQTT] Not connected, skipping publish...");
    }
  }
}
```

Listing 2: Firmware ESP32 — main.cpp

PHỤ LỤC B. Mã nguồn Node-RED (JSON Flow)

Các bước import flow vào Node-RED:

1. Truy cập Node-RED tại `http://127.0.0.1:1880`.
2. Nhấn Menu → Import.
3. Chọn thẻ Clipboard, paste toàn bộ nội dung JSON bên dưới.
4. Nhấn **Import** để thêm flow.
5. Nhấn **Deploy** (nút đỏ góc phải) để chạy flow.

Lưu ý: Sau khi import, kiểm tra và cập nhật cấu hình node **mqtt-broker**: hostname, port (8883), username, password phải khớp với tài khoản HiveMQ Cloud đang sử dụng.

```json
{
  "version": 1,
  "author": "AgriSense IoT",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp",
      "top": 0,
      "left": 91.24,
      "attrs": {}
    },
    {
      "type": "wokwi-dht22",
      "id": "dht1",
      "top": -220.5,
      "left": 263.4,
      "attrs": {
        "temperature": "25",
        "humidity": "55"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r_dht",
      "top": -185,
      "left": 220,
      "attrs": {
        "value": "10000"
      }
    },
    {
      "type": "wokwi-potentiometer",
      "id": "pot1",
      "top": -193.3,
      "left": -86.6,
      "attrs": {
        "value": "50000"
      }
    },
```

```
    {
      "type": "wokwi-buzzer",
      "id": "buzzer1",
      "top": 136.8,
      "left": -103.8,
      "attrs": {
        "volume": "0.1"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r_buzzer",
      "top": 312,
      "left": -29.35,
      "rotate": 90,
      "attrs": {
        "value": "220"
      }
    },
    {
      "type": "wokwi-bh1750",
      "id": "bh1750_1",
      "top": -20,
      "left": 450,
      "attrs": {
        "lux": "1500"
      }
    },
    {
      "type": "wokwi-relay-module",
      "id": "relay1",
      "top": 269,
      "left": 259.2,
      "attrs": {}
    }
  ],
  "connections": [
    [
      "esp:TX",
      "$serialMonitor:RX",
      "",
```

```
      []
    ],
    [
      "esp:RX",
      "$serialMonitor:TX",
      "",
      []
    ],
    [
      "esp:3V3",
      "dht1:VCC",
      "red",
      [
        "v-67.2",
        "h182.55"
      ]
    ],
    [
      "esp:GND.2",
      "dht1:GND",
      "black",
      []
    ],
    [
      "esp:3V3",
      "r_dht:1",
      "red",
      [
        "v-67.2",
        "h163.2",
        "v-57.6"
      ]
    ],
    [
      "r_dht:2",
      "dht1:DATA",
      "orange",
      [
        "h9.6",
        "v9.6"
      ]
```

```
    ],
    [
      "dht1:DATA",
      "esp:4",
      "yellow",
      [
        "h57.6"
      ]
    ],
    [
      "esp:3V3",
      "pot1:VCC",
      "red",
      [
        "h0.15",
        "v-67.2",
        "h-143.2"
      ]
    ],
    [
      "esp:GND.2",
      "pot1:GND",
      "black",
      [
        "v-38.4",
        "h-249.6"
      ]
    ],
    [
      "pot1:SIG",
      "esp:34",
      "blue",
      [
        "h-0.4",
        "v144"
      ]
    ],
    [
      "esp:27",
      "r_buzzer:1",
      "orange",
```

```
      [
        "h-95.85",
        "v153.6"
      ]
    ],
    [
      "r_buzzer:2",
      "buzzer1:2",
      "orange",
      []
    ],
    [
      "esp:GND.2",
      "buzzer1:1",
      "black",
      [
        "h76.8",
        "v-57.6",
        "h-384",
        "v249.6"
      ]
    ],
    [
      "esp:3V3",
      "bh1750_1:VCC",
      "red",
      []
    ],
    [
      "esp:GND.2",
      "bh1750_1:GND",
      "black",
      []
    ],
    [
      "esp:21",
      "bh1750_1:SDA",
      "green",
      []
    ],
    [
```

```
      "esp:22",
      "bh1750_1:SCL",
      "purple",
      []
    ],
    [
      "esp:26",
      "relay1:IN",
      "green",
      [
        "h-38.25",
        "v230.6"
      ]
    ],
    [
      "esp:5V",
      "relay1:VCC",
      "red",
      [
        "h-28.65",
        "v124.8"
      ]
    ],
    [
      "esp:GND.2",
      "relay1:GND",
      "black",
      [
        "h38.4",
        "v307.6"
      ]
    ]
  ],
  "dependencies": {}
}
```

Listing 3: Node-RED Flow (JSON) — node-red.json

PHỤ LỤC C. Mã nguồn sơ đồ Wokwi (JSON)

Các bước mở sơ đồ mạch trên Wokwi:

1. Truy cập https://wokwi.com/.

2. Đăng nhập (hoặc tạo tài khoản mới).

3. Tạo project mới hoặc import file JSON bên dưới.

4. Nhấn Play để chạy mô phỏng.

```
{
  "version": 1,
  "author": "AgriSense IoT",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp",
      "top": 0,
      "left": 91.24,
      "attrs": {}
    },
    {
      "type": "wokwi-dht22",
      "id": "dht1",
      "top": -220.5,
      "left": 263.4,
      "attrs": {
        "temperature": "25",
        "humidity": "55"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r_dht",
      "top": -185,
      "left": 220,
      "attrs": {
        "value": "10000"
      }
    },
    {
      "type": "wokwi-potentiometer",
      "id": "pot1",
```

```
      "top": -193.3,
      "left": -86.6,
      "attrs": {
        "value": "50000"
      }
    },
    {
      "type": "wokwi-buzzer",
      "id": "buzzer1",
      "top": 136.8,
      "left": -103.8,
      "attrs": {
        "volume": "0.1"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r_buzzer",
      "top": 312,
      "left": -29.35,
      "rotate": 90,
      "attrs": {
        "value": "220"
      }
    },
    {
      "type": "wokwi-bh1750",
      "id": "bh1750_1",
      "top": -20,
      "left": 450,
      "attrs": {
        "lux": "1500"
      }
    },
    {
      "type": "wokwi-relay-module",
      "id": "relay1",
      "top": 269,
      "left": 259.2,
      "attrs": {}
    }
```

```
  ],
  "connections": [
    [
      "esp:TX",
      "$serialMonitor:RX",
      "",
      []
    ],
    [
      "esp:RX",
      "$serialMonitor:TX",
      "",
      []
    ],
    [
      "esp:3V3",
      "dht1:VCC",
      "red",
      [
        "v-67.2",
        "h182.55"
      ]
    ],
    [
      "esp:GND.2",
      "dht1:GND",
      "black",
      []
    ],
    [
      "esp:3V3",
      "r_dht:1",
      "red",
      [
        "v-67.2",
        "h163.2",
        "v-57.6"
      ]
    ],
    [
      "r_dht:2",
```

```
      "dht1:DATA",
      "orange",
      [
        "h9.6",
        "v9.6"
      ]
    ],
    [
      "dht1:DATA",
      "esp:4",
      "yellow",
      [
        "h57.6"
      ]
    ],
    [
      "esp:3V3",
      "pot1:VCC",
      "red",
      [
        "h0.15",
        "v-67.2",
        "h-143.2"
      ]
    ],
    [
      "esp:GND.2",
      "pot1:GND",
      "black",
      [
        "v-38.4",
        "h-249.6"
      ]
    ],
    [
      "pot1:SIG",
      "esp:34",
      "blue",
      [
        "h-0.4",
        "v144"
```

```
      ]
    ],
    [
      "esp:27",
      "r_buzzer:1",
      "orange",
      [
        "h-95.85",
        "v153.6"
      ]
    ],
    [
      "r_buzzer:2",
      "buzzer1:2",
      "orange",
      []
    ],
    [
      "esp:GND.2",
      "buzzer1:1",
      "black",
      [
        "h76.8",
        "v-57.6",
        "h-384",
        "v249.6"
      ]
    ],
    [
      "esp:3V3",
      "bh1750_1:VCC",
      "red",
      []
    ],
    [
      "esp:GND.2",
      "bh1750_1:GND",
      "black",
      []
    ],
    [
```

```
      "esp:21",
      "bh1750_1:SDA",
      "green",
      []
    ],
    [
      "esp:22",
      "bh1750_1:SCL",
      "purple",
      []
    ],
    [
      "esp:26",
      "relay1:IN",
      "green",
      [
        "h-38.25",
        "v230.6"
      ]
    ],
    [
      "esp:5V",
      "relay1:VCC",
      "red",
      [
        "h-28.65",
        "v124.8"
      ]
    ],
    [
      "esp:GND.2",
      "relay1:GND",
      "black",
      [
        "h38.4",
        "v307.6"
      ]
    ]
  ],
  "dependencies": {}
}
```

Listing 4: Sơ đồ mạch Wokwi (JSON) — diagram.json